

INDEX

Practical No	Practical Aim	Date	Sign
1	Design a bot using AIML.	26-09-2023	
2	Design an Expert system using AIML.	10-10-2023	
3	Implement Bayes Theorem using Python	17-10-2023	
	A] Consider a human population that may or may not have cancer (Cancer is True or False) and a medical test that returns positive or negative for detecting cancer (Test is Positive or Negative), e.g., like a mammogram for detecting breast cancer.	17-10-2023	
	B] A bag I contains 4 white and 6 black balls while another Bag II contains 4 white and 3 black balls. One ball is drawn at random from one of the bags, and it is found to be black. Find the probability that it was drawn from Bag I.	17-10-2023	
	C] Imagine there is a drug test that is 98% accurate, meaning that 98% of the time, it shows a true positive result for someone using the drug, and 98% of the time, it shows a true negative result for nonusers of the drug.	17-10-2023	
4	Implement Conditional Probability and joint probability using Python.	07-11-2023	
5	Write a program for to implement Rule based system. (Prolog)	20-11-2023	
6	Design a Fuzzy based application using Python / R.	23-11-2023	
A	Solve Tipping problem using fuzzy logic.		
7	Write an application to simulate supervised and unsupervised learning model.	30-11-2023	
	A] Supervised Learning (iris_data)		
	B] Supervised Learning (diabetes_data)		
	C] Un-supervised Learning (Kmeans_iris_data)		
8	Write an application to implement clustering algorithm.	01-12-2023	
9	Write an application to implement support vector machine algorithm.	08-12- 2023	
10	Write a program to implement Backpropagation Algorithm.	20-12-2023	

Practical No: 1

Aim: Design an Expert system using AIML.

A. An expert system for responding the patient query for identifying the flu.

Code:

- std-startup1.xml

```
<aiml version="1.0.1" encoding="UTF-8">
  <!-- std-startup.xml -->
  <!-- Category is an atomic AIML unit -->
  <category>
    <!-- Pattern to match in user input -->
    <!-- If user enters "LOAD AIML B" -->
    <pattern>LOAD AIML B</pattern>
    <!-- Template is the response to the pattern -->
    <!-- This learn an aiml file -->
    <template>
      <learn>basic_chat1.aiml</learn>
      <!-- You can add more aiml files here -->
      <!--<learn>more_aiml.aiml</learn>-->
    </template>
  </category>
</aiml>
```

- basic_chat1.aiml

```
<aiml version="1.0.1" encoding="UTF-8">
<!-- basic_chat.aiml -->
<category>
  <pattern>HELLO</pattern>
  <template>
    WHAT WOULD YOU LIKE TO DISCUSS? : HEALTH, MOVIES
  </template>
</category>

<category>
  <pattern>MOVIES</pattern>
  <template>
    YES <set name = "topic">MOVIES</set>
  </template>
</category>

<category>
  <pattern>HEALTH</pattern>
```

```
<template>
YES <set name = "topic">HEALTH</set>
</template>
</category>

<topic name ="MOVIES">
  <category>
    <pattern>*</pattern>
    <template>
      DO YOU LIKE COMEDY MOVIES?
    </template>
  </category>

  <category>
    <pattern>YES</pattern>
    <template>
      I TOO LIKE COMEDY MOVIES
    </template>
  </category>

  <category>
    <pattern>NO</pattern>
    <template>
      BUT I LIKE COMEDY MOVIES
    </template>
  </category>
</topic>

<topic name ="HEALTH">

  <category>
    <pattern>*</pattern>
    <template>
      DO YOU HAVE FEVER?
    </template>
  </category>

  <category>
    <pattern>YES</pattern>
    <template>
      PLEASE TAKE MEDICINES AND PROPER REST
    </template>
  </category>

  <category>
    <pattern>NO</pattern>
    <template>
      GO OUT FOR A WALK AND LISTEN MUSIC
    </template>
  </category>
```

</topic>

```
<category>
    <pattern>NICE TALKING TO YOU</pattern>
    <template>
        SAME HERE...!!
    </template>
</category>

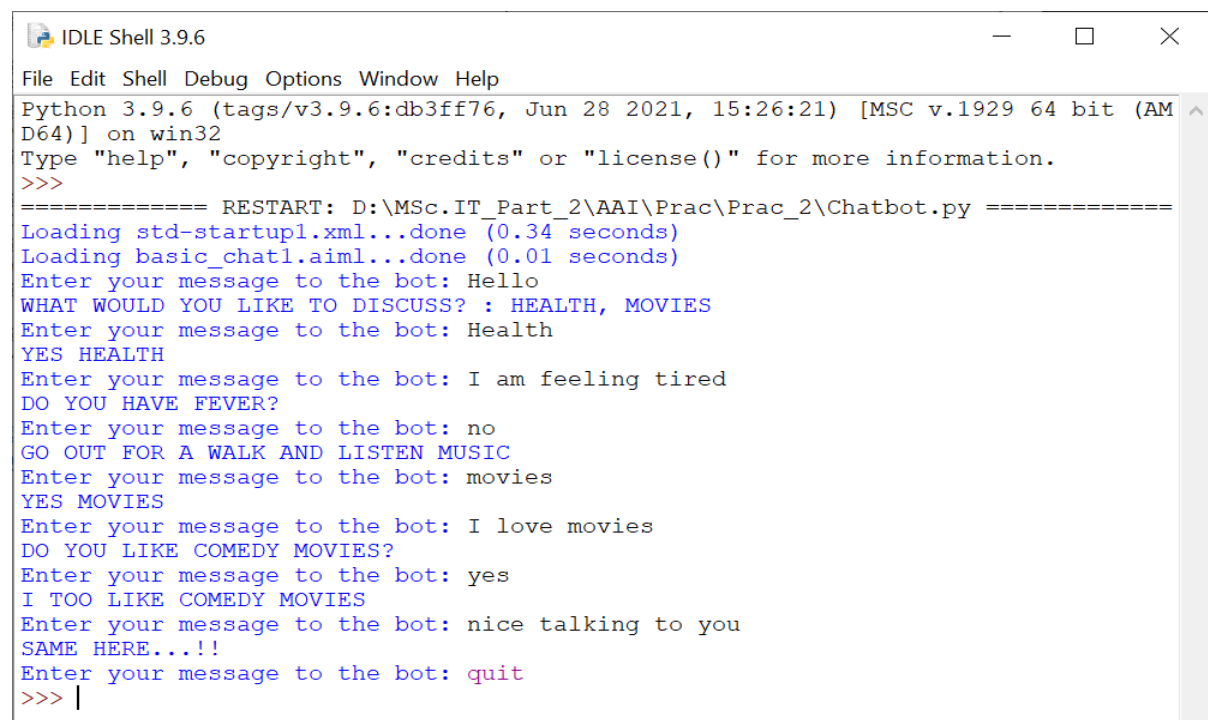
</aiml>
```

- chat_bot.py

```
import aiml
# Create the kernel and learn AIML files
kernel = aiml.Kernel()
kernel.learn("std-startup1.xml")
kernel.respond("load aiml b")

# Press CTRL-C to break this loop
while True:
    message = input("Enter your message to the bot: ")
    if message == "quit":
        break
    else:
        bot_response = kernel.respond(message)
        print(bot_response)
```

Output:



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\MSc.IT_Part_2\AAI\Prac\Prac_2\Chatbot.py =====
Loading std-startup1.xml...done (0.34 seconds)
Loading basic_chat1.aiml...done (0.01 seconds)
Enter your message to the bot: Hello
WHAT WOULD YOU LIKE TO DISCUSS? : HEALTH, MOVIES
Enter your message to the bot: Health
YES HEALTH
Enter your message to the bot: I am feeling tired
DO YOU HAVE FEVER?
Enter your message to the bot: no
GO OUT FOR A WALK AND LISTEN MUSIC
Enter your message to the bot: movies
YES MOVIES
Enter your message to the bot: I love movies
DO YOU LIKE COMEDY MOVIES?
Enter your message to the bot: yes
I TOO LIKE COMEDY MOVIES
Enter your message to the bot: nice talking to you
SAME HERE...!!
Enter your message to the bot: quit
>>> |
```

Practical No: 2

Aim: Design a bot using AIML.

Code:

- std-startup.xml

```
<aiml version="1.0.1" encoding="UTF-8">
  <!-- std-startup.xml -->
  <!-- Category is an atomic AIML unit -->
  <category>
    <!-- Pattern to match in user input -->
    <!-- If user enters "LOAD AIML B" -->
    <pattern>LOAD AIML B</pattern>
    <!-- Template is the response to the pattern -->
    <!-- This learn an aiml file -->
    <template>
      <learn>basic_chat.aiml</learn>
      <!-- You can add more aiml files here -->
      <!--<learn>more_aiml.aiml</learn>-->
    </template>
  </category>
</aiml>
```

- basic_chat.aiml

```
<aiml version="1.0.1" encoding="UTF-8">
<!-- basic_chat.aiml -->

<category>
  <pattern>HELLO</pattern>
  <template>
    Well, hello!
  </template>
</category>

<category>
  <pattern>WHAT ARE YOU</pattern>
  <template>
    I'm a bot, silly!
  </template>
</category>

<category>
  <pattern>MY NAME IS *</pattern>
  <template>
```

```
<set name = "username"><star/></set> is the nice name.

</template>
</category>

<category>
  <pattern>I LIKE *</pattern>
  <template>

    <set name = "liking"><star/></set> is also my favourite.

  </template>
</category>

<category>
  <pattern>MY DOG NAME IS *</pattern>
  <template>

    THAT IS INTERESTING THAT YOU HAVE A DOG NAMED <set name
    ="dog"><star/></set>.

  </template>
</category>

<category>
  <pattern>Bye</pattern>
  <template>

    Bye!!! <get name = "username"/> Thanks for talking with me.

  </template>
</category>
</aiml>
```

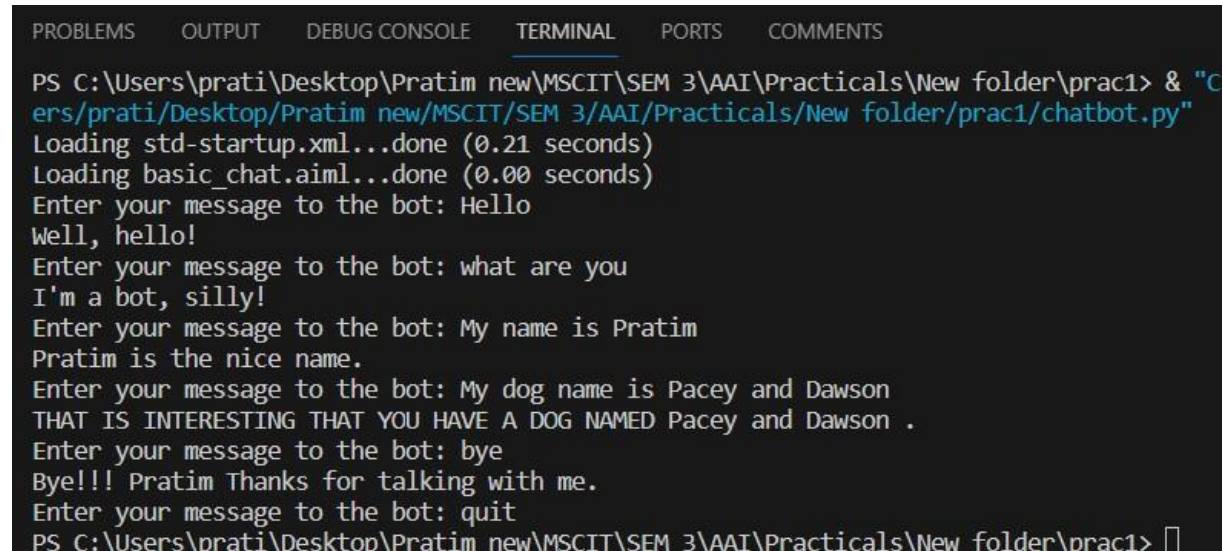
- chat_bot.py

```
import aiml
import time
# Create the kernel and learn AIML files
kernel = aiml.Kernel()
kernel.learn("std-startup.xml")
kernel.respond("load aiml b")

# Press CTRL-C to break this loop
while True:
    message = input("Enter your message to the bot: ")
    if message == "quit":
        break
    else:
```

```
bot_response = kernel.respond(message)
print(bot_response)
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS
PS C:\Users\prati\Desktop\Pratim new\MSCIT\SEM 3\AAI\Practicals\New folder\prac1> & "C:\Users\prati\Desktop\Pratim new\MSCIT\SEM 3\AAI\Practicals\New folder\prac1\chatbot.py"
Loading std-startup.xml...done (0.21 seconds)
Loading basic_chat.aiml...done (0.00 seconds)
Enter your message to the bot: Hello
well, hello!
Enter your message to the bot: what are you
I'm a bot, silly!
Enter your message to the bot: My name is Pratim
Pratim is the nice name.
Enter your message to the bot: My dog name is Pacey and Dawson
THAT IS INTERESTING THAT YOU HAVE A DOG NAMED Pacey and Dawson .
Enter your message to the bot: bye
Bye!!! Pratim Thanks for talking with me.
Enter your message to the bot: quit
PS C:\Users\prati\Desktop\Pratim new\MSCIT\SEM 3\AAI\Practicals\New folder\prac1> █
```

Practical No: 3

Aim: Implement Bayes Theorem using Python.

A. Consider a human population that may or may not have cancer (Cancer is True or False) and a medical test that returns positive or negative for detecting cancer (Test is Positive or Negative), e.g., like a mammogram for detecting breast cancer. If a randomly selected patient has the test and it comes back positive, what is the probability that the patient has cancer?

Code:

```
# calculate P(A|B) given P(A), P(B|A), P(B|not A)
def bayes_theorem(p_a, p_b_given_a, p_b_given_not_a):
    # calculate P(not A)
    not_a = 1 - p_a
    # calculate P(B)
    p_b = p_b_given_a * p_a + p_b_given_not_a * not_a
    # calculate P(A|B)
    p_a_given_b = (p_b_given_a * p_a) / p_b
    return p_a_given_b
# P(A)
p_a = 0.0002
# P(B|A)
p_b_given_a = 0.85
# P(B|not A)
p_b_given_not_a = 0.05
# calculate P(A|B)
result = bayes_theorem(p_a, p_b_given_a, p_b_given_not_a)
# summarize
print('P(A|B) = %.3f%%' % (result * 100))
```

Output:

P(A|B) = 0.339%

- B. A bag I contains 4 white and 6 black balls while another Bag II contains 4 white and 3 black balls. One ball is drawn at random from one of the bags, and it is found to be black. Find the probability that it was drawn from Bag I.

Code:

```
def bayes_theorem(P_E1, P_E2, P_A_given_E1, P_A_given_E2):  
  
    P_A= P_A_given_E1* P_E1 + P_A_given_E2 * P_E2  
  
    P_E1_given_A = (P_A_given_E1* P_E1) / P_A  
  
    return P_E1_given_A  
  
P_E1 = 0.5  
P_E2 = 0.5  
  
P_A_given_E1= 0.6  
  
P_A_given_E2 = 0.42  
  
result = bayes_theorem(P_E1, P_E2, P_A_given_E1, P_A_given_E2)  
  
print('P(E1|A) = %.3f%%' % (result * 100))
```

Output:

```
P(E1|A) = 58.824%
```

- C. Imagine there is a drug test that is 98% accurate, meaning that 98% of the time, it shows a true positive result for someone using the drug, and 98% of the time, it shows a true negative result for nonusers of the drug.

Code:

```
def bayes_theorem(p_H, p_E_given_H):  
  
    not_H = 1 - p_H  
  
    p_E_given_not_H = 1 - p_E_given_H  
  
    p_E = p_E_given_H * p_H + p_E_given_not_H * not_H  
  
    p_H_given_E = (p_E_given_H * p_H) / p_E  
  
    return p_H_given_E  
p_H = 0.005  
  
p_E_given_H = 0.98  
  
result = bayes_theorem(p_H, p_E_given_H)  
  
# summarize  
print('P(H|E) = %.3f%%' % (result * 100))
```

Output:

```
P(H|E) = 19.758%
```

Practical No: 4

Aim: Implement Conditional Probability and joint probability using Python.

Code:

```
import enum, random
class Kid(enum.Enum):
    BOY = 0
    GIRL = 1
def random_kid() -> Kid:
    return random.choice([Kid.BOY, Kid.GIRL])
both_girls = 0
older_girl = 0
either_girl = 0
random.seed(0)
for _ in range(10000):
    younger = random_kid()
    older = random_kid()
    if older == Kid.GIRL:
        older_girl += 1
    if older == Kid.GIRL and younger == Kid.GIRL:
        both_girls += 1
    if older == Kid.GIRL or younger == Kid.GIRL:
        either_girl += 1
print("P(both | older):", both_girls / older_girl) # 0.5007089325501317
print("P(both | either):", both_girls / either_girl) # 0.3311897106109325
```

Output:

```
P(both | older): 0.5007089325501317
P(both | either): 0.3311897106109325
```

Practical No: 5

Aim: Write a program for to implement Rule based system.

Code:

```
go:-
    hypothesis(Disease),
    write('I believe that the patient have '),
    write(Disease),
    nl,
    write('TAKE CARE '),
    undo.

/*Hypothesis that should be tested*/
hypothesis(cold) :- cold, !.
hypothesis(flu) :- flu, !.
hypothesis(typhoid) :- typhoid, !.
hypothesis(measles) :- measles, !.
hypothesis(malaria) :- malaria, !.
hypothesis(unknown). /* no diagnosis*/

/*Hypothesis Identification Rules*/
cold :-
    verify(headache),
    verify(runny_nose),
    verify(sneezing),
    verify(sore_throat),
    write('Advices and Sugestions:'),
    nl,
    write('1: Tylenol/tab'),
    nl,
    write('2: panadol/tab'),
    nl,
    write('3: Nasal spray'),
    nl,
    write('Please wear warm cloths Because'),
    nl.

flu :-
    verify(fever),
    verify(headache),
    verify(chills),
    verify(body_ache),
    write('Advices and Sugestions:'),
    nl,
    write('1: Tamiflu/tab'),
```

```
nl,  
write('2: panadol/tab'),  
nl,  
write('3: Zanamivir/tab'),  
nl,  
write('Please take a warm bath and do salt gargling Because'),  
nl.
```

typhoid :-

```
verify(headache),  
verify(abdominal_pain),  
verify(poor_appetite),  
verify(fever),  
write('Advices and Sugestions:'),  
nl,  
write('1: Chloramphenicol/tab'),  
nl,  
write('2: Amoxicillin/tab'),  
nl,  
write('3: Ciprofloxacin/tab'),  
nl,  
write('4: Azithromycin/tab'),  
nl,  
write('Please do complete bed rest and take soft Diet Because'),  
nl.
```

measles :-

```
verify(fever),  
verify(runny_nose),  
verify(rash),  
verify(conjunctivitis),  
write('Advices and Sugestions:'),  
nl,  
write('1: Tylenol/tab'),  
nl,  
write('2: Aleve/tab'),  
nl,  
write('3: Advil/tab'),  
nl,  
write('4: Vitamin A'),  
nl,  
write('Please Get rest and use more liquid Because'),  
nl.
```

malaria :-

```
verify(fever),  
verify(sweating),  
verify(headache),  
verify(nausea),  
verify(vomiting),
```

```
verify(diarrhea),
write('Advices and Sugestions:'),
nl,
write('1: Aralen/tab'),
nl,
write('2: Qualaquin/tab'),
nl,
write('3: Plaquenil/tab'),
nl,
write('4: Mefloquine'),
nl,
write('Please do not sleep in open air and cover your full skin Because'),
nl.

/* how to ask questions */
ask(Question) :-
    write('Does the patient have following symptom:'),
    write(Question),
    write('? '),
    read(Response),
    nl,
    ( (Response == yes ; Response == y)
    ->
    assert(yes(Question)) ;
    assert(no(Question)), fail).
:- dynamic yes/1,no/1.
/*How to verify something */
verify(S) :-
    (yes(S)
    ->
    true ;
    (no(S)
    ->
    fail ;
    ask(S))).

/* undo all yes/no assertions*/
undo :- retract(yes(_)),fail.
undo :- retract(no(_)),fail.
undo.
```

Output:

 SWI-Prolog (AMD64, Multi-threaded, version 9.0.4)

File Edit Settings Run Debug Help

Welcome to SWI-Prolog (threaded, 64 bits, version 9.0.4)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use ?- help(Topic). or ?- apropos(Word).

```
?-
% d:/MSc.IT_Part_2/AAI/Prac/Prolog_Prac/Prac_5.pl compiled 0.00 sec, 17 clauses
?- go.
Does the patient have following symptom:headache? yes.

Does the patient have following symptom:runny_nose? |: yes.

Does the patient have following symptom:sneezing? |: yes.

Does the patient have following symptom:sore_throat? |: yes.

Advices and Sugestions:
1: Tylenol/tab
2: panadol/tab
3: Nasal spray
Please wear warm cloths Because
I believe that the patient have cold
TAKE CARE
true.
```

```
?- ■

% d:/MSc.IT_Part_2/AAI/Prac/Prolog_Prac/Prac_5.pl compiled 0.00 sec, 17 clauses
?- go.
Does the patient have following symptom:headache? yes.

Does the patient have following symptom:runny_nose? |: yes.

Does the patient have following symptom:sneezing? |: no.

Does the patient have following symptom:fever? |: yes.

Does the patient have following symptom:chills? |: yes.

Does the patient have following symptom:body_ache? |: no.

Does the patient have following symptom:abdominal_pain? |: no.

Does the patient have following symptom:rash? |: yes.

Does the patient have following symptom:conjunctivitis? |: yes.

Advices and Sugestions:
1: Tylenol/tab
2: Aleve/tab
3: Advil/tab
4: Vitamin A
Please Get rest and use more liquid Because
I believe that the patient have measles
TAKE CARE
true.
```

```
?- ■
```

```
?- go.  
Does the patient have following symptom:headache? yes.  
  
Does the patient have following symptom:runny_nose? |: no.  
  
Does the patient have following symptom:fever? |: yes.  
  
Does the patient have following symptom:chills? |: yes.  
  
Does the patient have following symptom:body_ache? |: yes.
```

Advices and Sugestions:

```
1: Tamiflu/tab  
2: panadol/tab  
3: Zanamivir/tab  
Please take a warm bath and do salt gargling Because  
I believe that the patient have flu  
TAKE CARE  
true.
```

```
?- hypothesis(cold).  
Does the patient have following symptom:headache? yes.  
  
Does the patient have following symptom:runny_nose? |: yes.  
  
Does the patient have following symptom:sneezing? |: yes.  
  
Does the patient have following symptom:sore_throat? |: yes.
```

Advices and Sugestions:

```
1: Tylenol/tab  
2: panadol/tab  
3: Nasal spray  
Please wear warm cloths Because  
true.
```


Practical No: 6

Aim: Design a Fuzzy based application using Python / R.

A. Solve Tipping problem using fuzzy logic.

Theory:

The ‘tipping problem’ is commonly used to illustrate the power of fuzzy logic principles to generate complex behaviour from a compact, intuitive set of expert rules. If you are new to the world of fuzzy control systems, you might want to check out the Fuzzy Control Primer before reading through this worked example.

The Tipping Problem

Let us create a fuzzy control system which models how you might choose to tip at a restaurant. When tipping, you consider the service and food quality, rated between 0 and 10. You use this to leave a tip of between 0 and 25%.

We would formulate this problem as:

- **Antecedents (Inputs)**

- service

- * Universe (i.e., crisp value range): How good was the service of the wait staff, on a scale of 0 to 10?

- * Fuzzy set (i.e., fuzzy value range): poor, acceptable, amazing

- food quality

- * Universe: How tasty was the food, on a scale of 0 to 10?

- * Fuzzy set: bad, decent, great

- **Consequents (Outputs)**

- tip

- * Universe: How much should we tip, on a scale of 0% to 25%

- * Fuzzy set: low, medium, high

- **Rules**

- IF the service was good or the food quality was good, THEN the tip will be high.

- IF the service was average, THEN the tip will be medium.

- IF the service was poor and the food quality was poor THEN the tip will be low.

- **Usage**

- If I tell this controller that I rated:

- * The service as 9.8, and

- * The quality as 6.5,

– it would recommend I leave:
* a 20.2% tip.

Code:

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

quality = ctrl.Antecedent(np.arange(0, 11, 1), 'quality')
service = ctrl.Antecedent(np.arange(0, 11, 1), 'service')
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')

quality.automf(3)
service.automf(3)

tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13])
tip['medium'] = fuzz.trimf(tip.universe, [0, 13, 25])
tip['high'] = fuzz.trimf(tip.universe, [13, 25, 25])

quality['average'].view()
service.view()
tip.view()

rule1 = ctrl.Rule(quality['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium'])
rule3 = ctrl.Rule(service['good'] | quality['good'], tip['high'])

rule1.view()

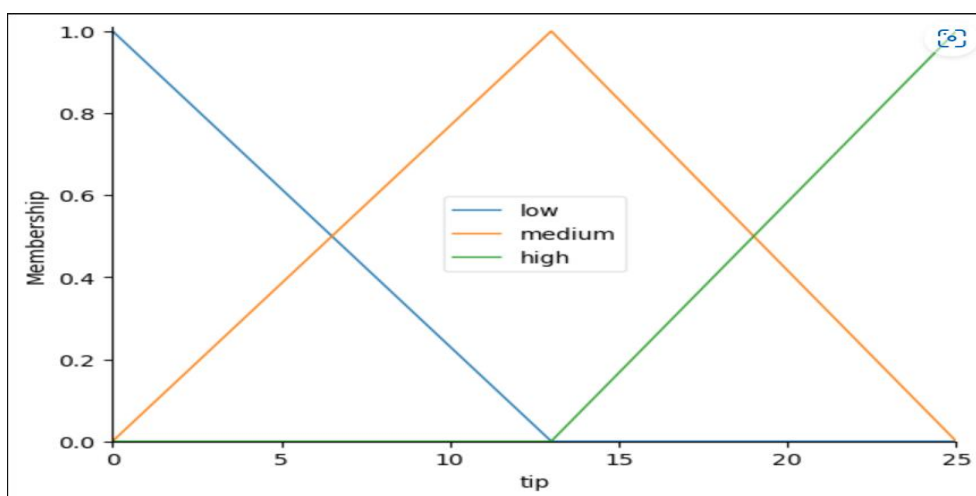
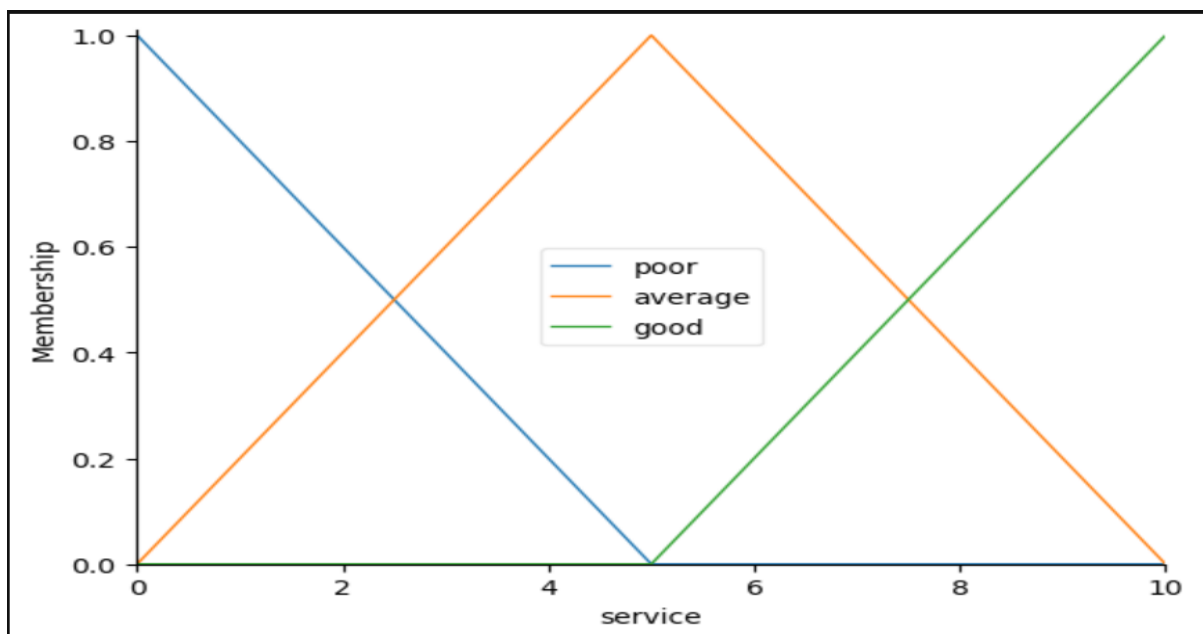
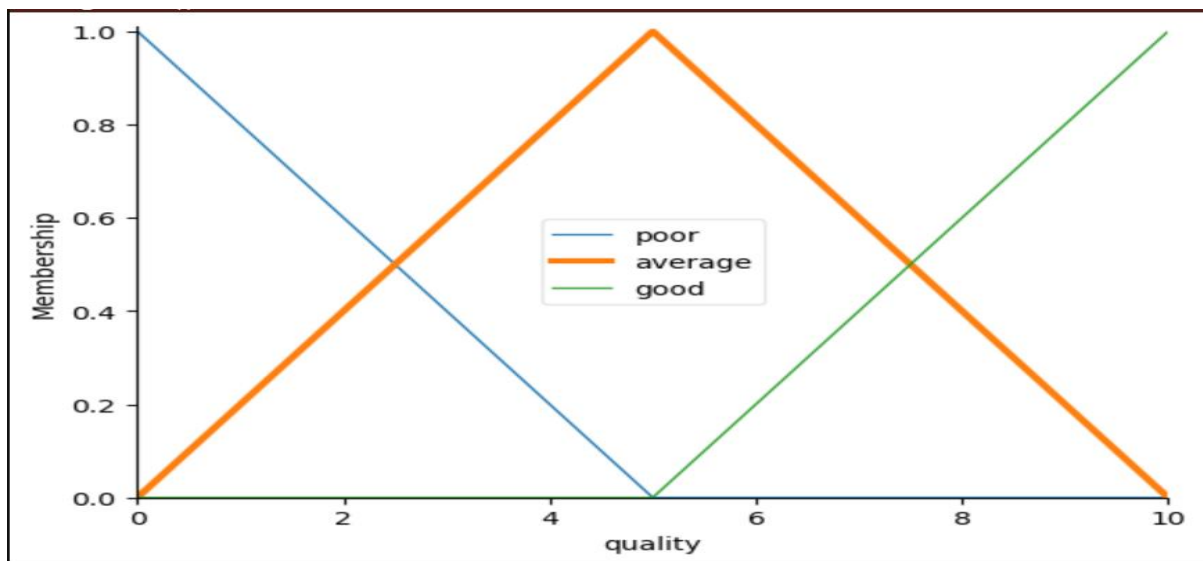
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
tipping = ctrl.ControlSystemSimulation(tipping_ctrl)

tipping.input['quality'] = 6.5
tipping.input['service'] = 9.8

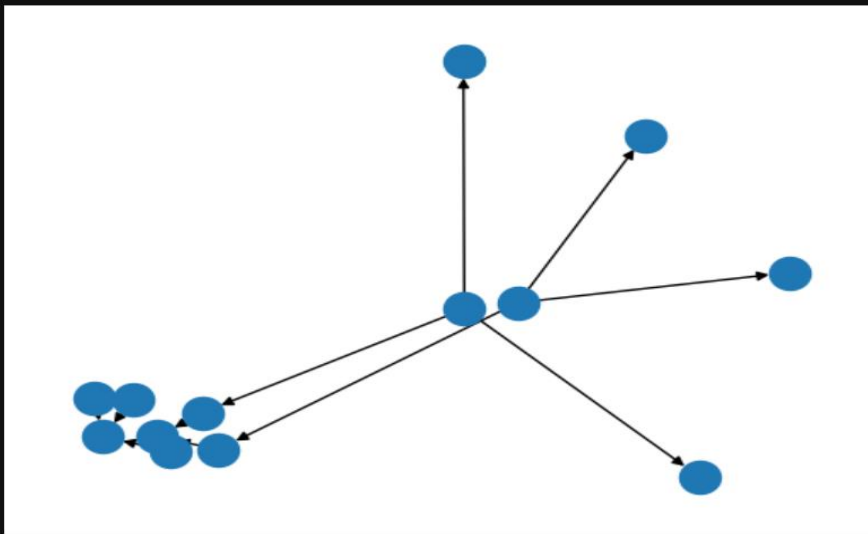
tipping.compute()

print(tipping.output['tip'])
tip.view(sim=tipping)
```

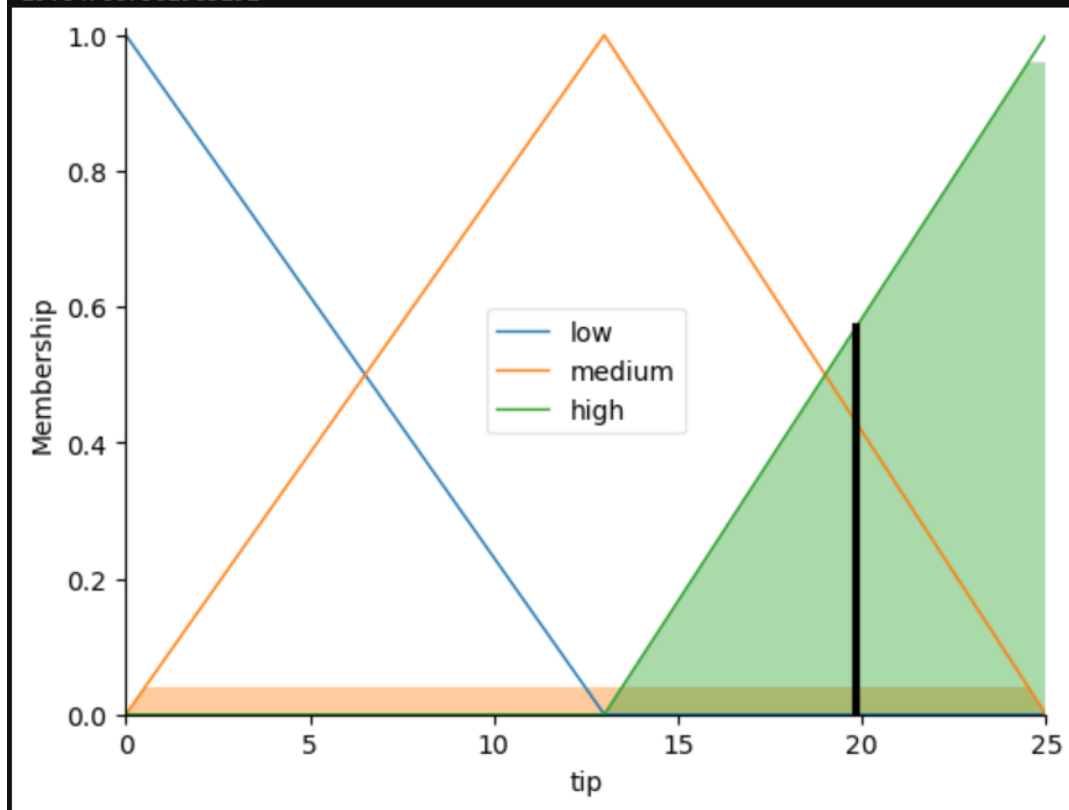
Output:



[9]: (<Figure size 640x480 with 1 Axes>, <Axes: >)



19.847607361963192



Practical No: 7

Aim: Write an application to simulate supervised and un-supervised learning model.

- **Supervised Learning.**

A.

Code:

```
from sklearn import datasets
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
```

```
iris = datasets.load_iris()
```

```
type(iris)
```

```
iris.keys()
```

```
type(iris.data), type(iris.target)
```

```
iris.data.shape
```

```
iris.target_names
```

```
X = iris.data
```

```
# Load iris target set
```

```
Y = iris.target
```

```
# Convert datasets' type into dataframe
```

```
df = pd.DataFrame(X, columns=iris.feature_names)
```

```
df
```

```
knn = KNeighborsClassifier(n_neighbors=6)
```

```
knn.fit(iris['data'], iris['target'])
```

```
X = [
```

```
    [5.9, 1.0, 5.1, 1.8],
```

```
    [3.4, 2.0, 1.1, 4.8],
```

```
]
```

```
X
```

```
prediction = knn.predict(X)
prediction
```

Output:

```
[32]: sklearn.utils._bunch.Bunch
```

```
[33]: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

```
[34]: (numpy.ndarray, numpy.ndarray)
```

```
[35]: (150, 4)
```

```
[36]: array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

```
[37]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

```
[40]: [[5.9, 1.0, 5.1, 1.8], [3.4, 2.0, 1.1, 4.8]]
```

```
[41]: array([1, 1])
```

B. Supervised Learning (diabetes_data)

Code:

```
from sklearn import datasets, linear_model
import matplotlib.pyplot as plt
import numpy as np
diabetes = datasets.load_diabetes()
diabetes
diabetes_X = diabetes.data[:, np.newaxis, 2]
diabetes_X_train = diabetes_X[:-20]
diabetes_X_test = diabetes_X[-20:]
diabetes_y_train = diabetes.target[:-20]
diabetes_y_test = diabetes.target[-20:]
regr = linear_model.LinearRegression()
regr.fit(diabetes_X_train, diabetes_y_train)
print('Input Values')
diabetes_X_test
diabetes_y_pred = regr.predict(diabetes_X_test)
print("Predicted Output Values")
diabetes_y_pred
plt.scatter(diabetes_X_test, diabetes_y_test, color='black')
plt.plot(diabetes_X_test, diabetes_y_pred, color='red', linewidth=1)
plt.show()
```

Output:

```
[2]: {'data': array([[ 0.03807591,  0.05068012,  0.06169621, ..., -0.00259226,
  0.01990749, -0.01764613],
 [-0.00188202, -0.04464164, -0.05147406, ..., -0.03949338,
 -0.06833155, -0.09220405],
 [ 0.08529891,  0.05068012,  0.04445121, ..., -0.00259226,
  0.00286131, -0.02593034],
 ...,
 [ 0.04170844,  0.05068012, -0.01590626, ..., -0.01107952,
 -0.04688253,  0.01549073],
 [-0.04547248, -0.04464164,  0.03906215, ...,  0.02655962,
  0.04452873, -0.02593034],
 [-0.04547248, -0.04464164, -0.0730303 , ..., -0.03949338,
 -0.00422151,  0.00306441]]),
 'target': array([151.,  75., 141., 206., 135.,  97., 138.,  63., 110., 310., 101.,
  69., 179., 185., 118., 171., 166., 144.,  97., 168.,  68.,  49.,
  68., 245., 184., 202., 137.,  85., 131., 283., 129.,  59., 341.,
  87.,  65., 102., 265., 276., 252.,  90., 100.,  55.,  61.,  92.,
 259.,  53., 190., 142.,  75., 142., 155., 225.,  59., 104., 182.,
 128.,  52.,  37., 170., 170.,  61., 144.,  52., 128.,  71., 163.,
 150.,  97., 160., 178.,  48., 270., 202., 111.,  85.,  42., 170.,
 200., 252., 113., 143.,  51.,  52., 210.,  65., 141.,  55., 134.,
  42., 111.,  98., 164.,  48.,  96.,  90., 162., 150., 279.,  92.,
  83., 128., 102., 302., 198.,  95.,  53., 134., 144., 232.,  81.,
 104.,  59., 246., 297., 258., 229., 275., 281., 179., 200., 200.,
 173., 180.,  84., 121., 161.,  99., 109., 115., 268., 274., 158.,
 107.,  83., 103., 272.,  85., 280., 336., 281., 118., 317., 235.,
  60., 174., 259., 178., 128.,  96., 126., 288.,  88., 292.,  71.,
 197., 186.,  25.,  84.,  96., 195.,  53., 217., 172., 131., 214.,
```

```
'feature_names': ['age',  
'sex',  
'bmi',  
'bp',  
's1',  
's2',  
's3',  
's4',  
's5',  
's6'],  
'data_filename': 'diabetes_data_raw.csv.gz',  
'target_filename': 'diabetes_target.csv.gz',  
'data_module': 'sklearn.datasets.data'}
```

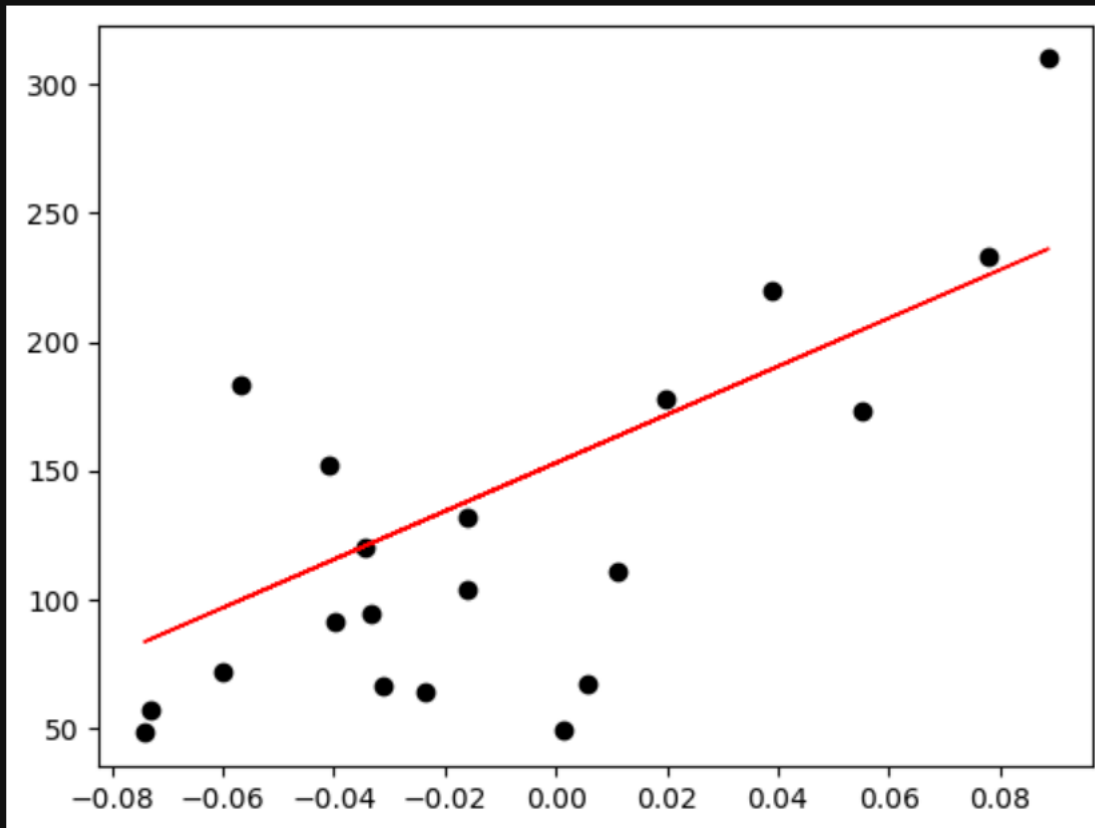
Input Values

```
array([[ 0.07786339],  
       [-0.03961813],  
       [ 0.01103904],  
       [-0.04069594],  
       [-0.03422907],  
       [ 0.00564998],  
       [ 0.08864151],  
       [-0.03315126],  
       [-0.05686312],  
       [-0.03099563],  
       [ 0.05522933],  
       [-0.06009656],  
       [ 0.00133873],  
       [-0.02345095],  
       [-0.07410811],  
       [ 0.01966154],  
       [-0.01590626],  
       [-0.01590626],  
       [ 0.03906215],  
       [-0.0730303 ]])
```

Predicted Output Values

```
[12]: array([225.9732401 , 115.74763374, 163.27610621, 114.73638965,  
            120.80385422, 158.21988574, 236.08568105, 121.81509832,  
            99.56772822, 123.83758651, 204.73711411, 96.53399594,  
            154.17490936, 130.91629517, 83.3878227 , 171.36605897,  
            137.99500384, 137.99500384, 189.56845268, 84.3990668 ])
```


[13]: [matplotlib.lines.Line2D at 0x7b925b0<]



- **Unsupervised Learning.**

Code:

```
from sklearn import datasets
from sklearn.cluster import KMeans
iris_df = datasets.load_iris()
model = KMeans(n_clusters=3)
model.fit(iris_df.data)
predicted_label = model.predict([[7.2, 3.5, 0.8, 1.6]])
all_predictions = model.predict(iris_df.data)
predicted_label
all_predictions
```

Output:

```
[7]: array([1])
```

```
[8]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          1, 1, 1, 1, 1, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 2, 2, 2, 2, 0, 2, 2,
          2, 2, 2, 0, 0, 2, 2, 2, 2, 0, 2, 0, 2, 0, 2, 2, 0, 0, 2, 2,
          2, 2,
          2, 0, 2, 2, 2, 2, 0, 2, 2, 2, 0, 2, 2, 0, 2, 2, 0])
```

Practical No: 8

Aim: Write an application to implement clustering algorithm.

Code:

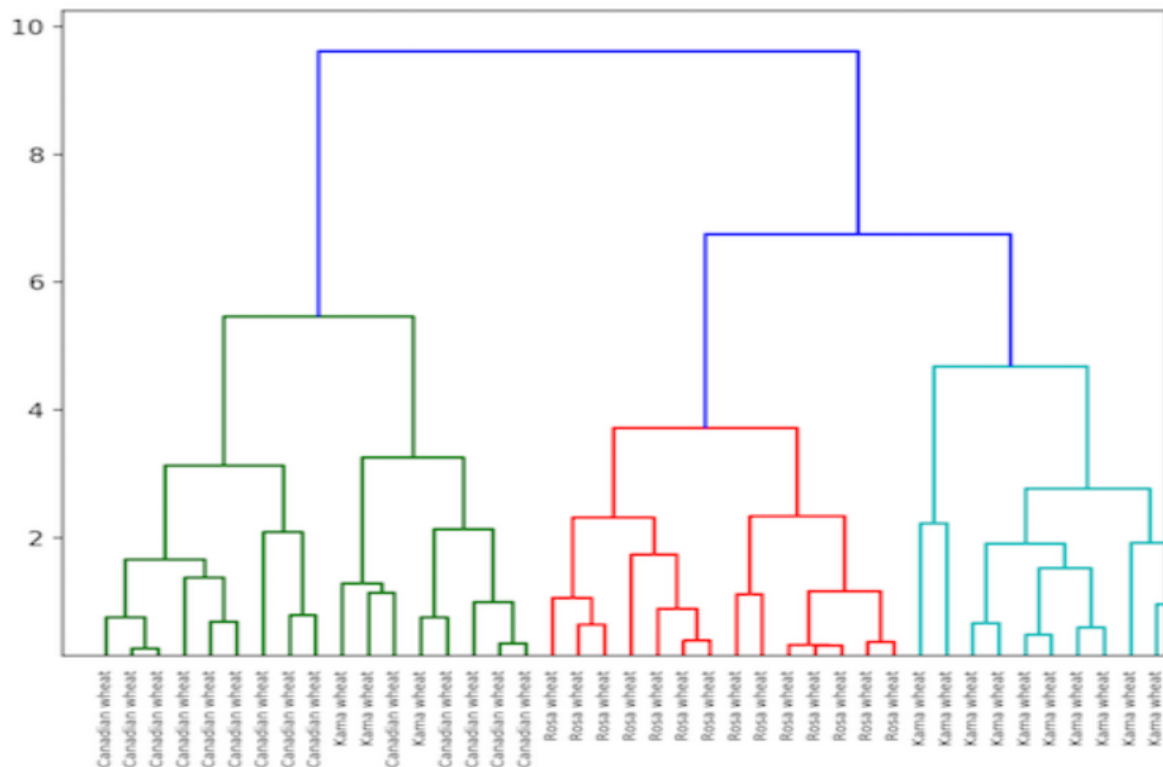
```
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt
import pandas as pd
```

```
seeds_df = pd.read_csv("seeds-less-rows.csv")
```

```
varieties = list(seeds_df.pop('grain_variety'))
samples = seeds_df.values
mergings = linkage(samples, method='complete')
dendrogram(mergings,
            labels=varieties,
            leaf_rotation=90,
            leaf_font_size=6,
            )
```

```
plt.show()
```

Output:



Practical No: 9

Aim: Write an application to implement support vector machine algorithm.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn import svm

# linear data
X = np.array([1, 5, 1.5, 8, 1, 9, 7, 8.7, 2.3, 5.5, 7.7, 6.1])
y = np.array([2, 8, 1.8, 8, 0.6, 11, 10, 9.4, 4, 3, 8.8, 7.5])

# show unclassified data
plt.scatter(X, y)
plt.show()

# shaping data for training the model
training_X = np.vstack((X, y)).T
print(np.vstack((X, y)).T)
training_y = [0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1]

# define the model
clf = svm.SVC(kernel='linear', C=1.0)

# train the model
clf.fit(training_X, training_y)

# get the weight values for the linear equation from the trained SVM model
w = clf.coef_[0]

# get the y-offset for the linear equation
a = -w[0] / w[1]

# make the x-axis space for the data points
XX = np.linspace(0, 13)

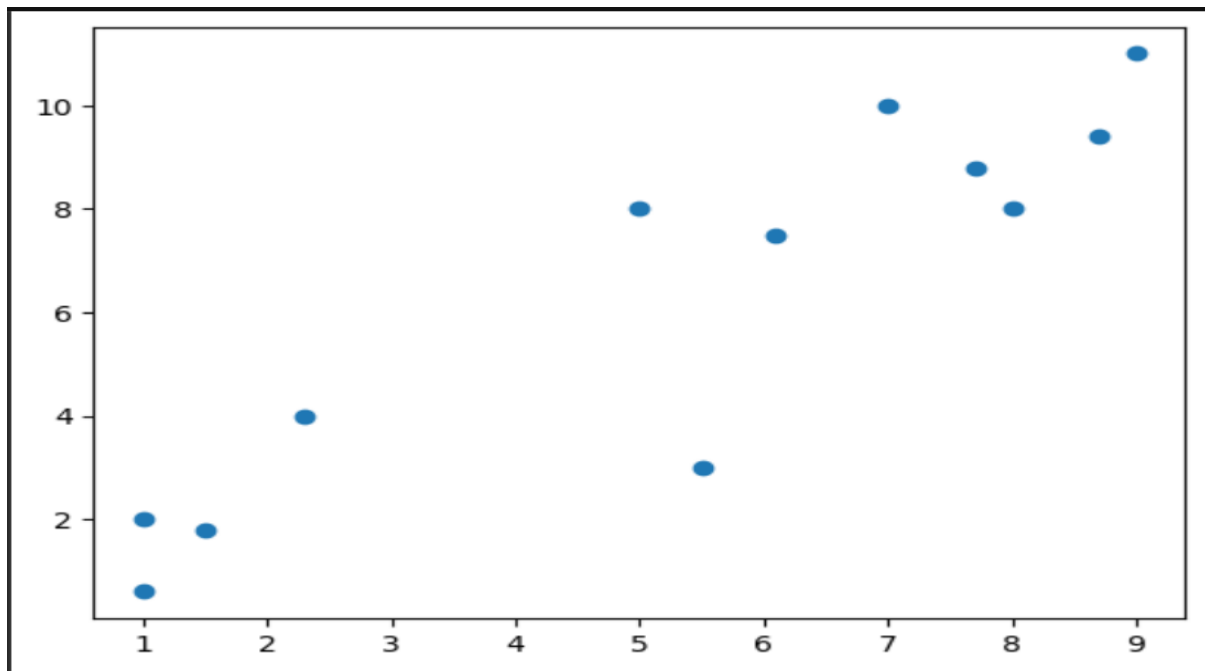
# get the y-values to plot the decision boundary
yy = a * XX - clf.intercept_[0] / w[1]

# plot the decision boundary
plt.plot(XX, yy, 'k-')

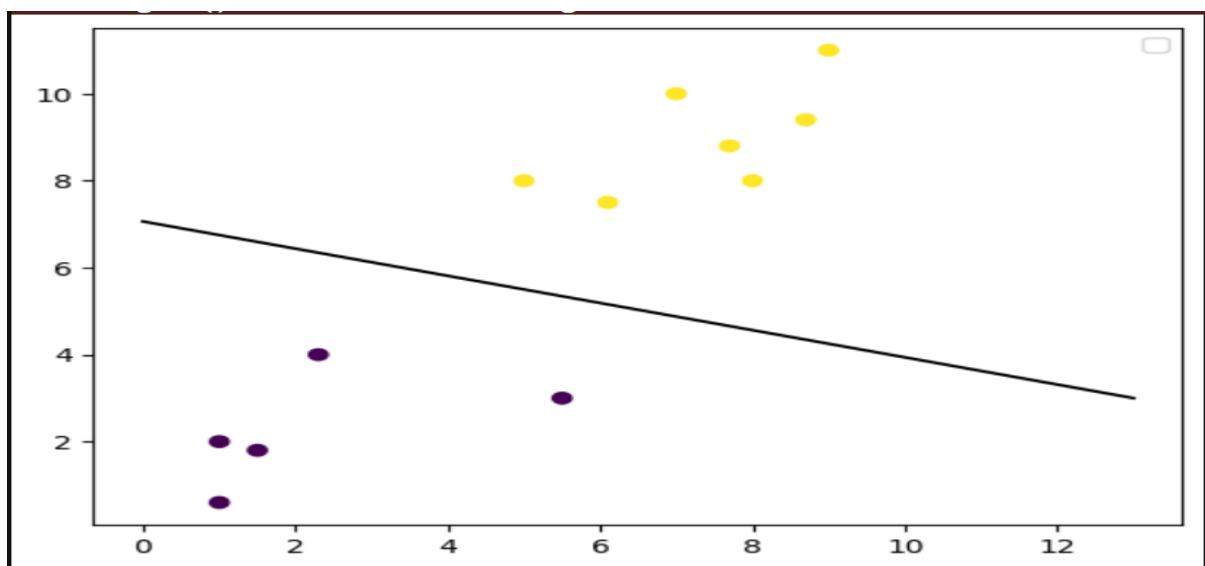
# show the plot visually
plt.scatter(training_X[:, 0], training_X[:, 1], c=training_y)
plt.legend()
```

plt.show()

Output:



```
[[ 1.  2. ]  
 [ 5.  8. ]  
 [ 1.5 1.8]  
 [ 8.  8. ]  
 [ 1.  0.6]  
 [ 9. 11. ]  
 [ 7. 10. ]  
 [ 8.7 9.4]  
 [ 2.3 4. ]  
 [ 5.5 3. ]  
 [ 7.7 8.8]  
 [ 6.1 7.5]]
```



Practical No: 10

Aim: Write a program to implement Backpropagation Algorithm.

```
import numpy as np
import decimal
import math
np.set_printoptions(precision=4)
v1=np.array([0.6,-0.3])
v2=np.array([-0.1,0.4])
w=np.array([-0.2,0.4,0.1])
b=np.array([0.3,0.5])
x=np.array([0,1])
alpha=0.25
zin=[]
print("calculate net input to z layer")
for i in range(0,2):
    zin.append(round(b[i]+x[0]*v1[i]+x[1]*v2[i],4))
print("zin=",zin)
z=[]
print("Apply activation function to calculate output")
for i in range(0,2):
    z.append(round(1/(1+math.exp(-zin[i])),4))
print("z=",z)
print("calculate net input to output layer")
yin=w[0]+z[0]*w[1]+z[1]*w[2]
print("yin=",yin)
print("calculate net output")
y=round(1/(1+math.exp(-yin)),4)
print("y=",y)
fyin=y*(1-y)
dk=round((1-y)*fyin,4)
print("dk",dk)
dw1=alpha*dk*z[0]
```

```
dw2=alpha*dk*z[1]
dw0=alpha*dk
print("compute error portion in delta")
din=[]
for i in range(1,3):
    din.append(dk*w[i])
print("din=",din)
fzin=[]
d=[]
print("error in delta")
for i in range(0,2):
    fzin.append(round(z[i] * (1-z[i]),4))
    d.append(round(din[i]*fzin[i],4))
print("fzin=",fzin)
print("d=",d)
dv=[[0,0],[0,0],[0,0]]
print("Change in weihts between input and hidden layer")
for i in range(0,3):
    for j in range(0,2):
        if i<=1:
            dv[i][j]=alpha*d[j]*x[i]
        else:
            dv[i][j]=alpha*d[j]
print(dv)
print("Final weights of network")
for i in range(0,2):
    v1[i]=v1[i]+dv[0][i]
print("v1=",v1)
for i in range(0,2):
    v2[i]=v2[i]+dv[1][i]
print("v2=",v2)
for i in range(0,2):
```

```
b[i]=b[i]+dv[2][i]
print("b=",b)
w[1]=w[1]+dw1
w[2]=w[2]+dw2
w[0]=w[0]+dw0
print("w=",w)
```

OUTPUT:-

```
===== RESTART: C:/Users/anike/OneDrive/Desktop/AAI prac/prac 10.py =====
calculate net input to z layer
zin= [0.2, 0.9]
Apply activation function to calculate output
z= [0.5498, 0.7109]
calculate net input to output layer
yin= 0.09101
calculate net output
y= 0.5227
dk 0.1191
compute error portion in delta
din= [0.04764, 0.01191]
error in delta
fzin= [0.2475, 0.2055]
d= [0.0118, 0.0024]
Change in weights between input and hidden layer
[[0.0, 0.0], [0.00295, 0.0006], [0.00295, 0.0006]]
Final weights of network
v1= [ 0.6 -0.3]
v2= [-0.0971  0.4006]
b= [0.3029 0.5006]
w= [-0.1702  0.4164  0.1212]
>> |
```